

LAPORAN PRAKTIKUM PBO 4

disusun Oleh:

Muhammad Faiz An Anri

NIM 2511532021

Dosen Pengampu: Nurfiah S.ST, M.KOM

Asisten Pratikum: Siti Fadhilah Rahmi



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

Kata Pengantar

Ucapan syukur kita lafalkan kepada Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah PBO.

Ucapan terima kasih disampaikan kepada Bapak Nurfiah, M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

Daftar Isi

BAB I

Pendahuluan

1. Latar Belakang

Pemrograman Berorientasi Objek (PBO) memungkinkan suatu sistem direpresentasikan melalui objek-objek yang saling berhubungan. Pada praktikum ini, konsep tersebut diterapkan melalui relasi antarobjek menggunakan Class Diagram dan Asosiasi one-to-many. Sistem perbankan yang sebelumnya hanya melakukan transaksi setor dan tarik saldo dikembangkan agar setiap aktivitas transaksi dapat direpresentasikan sebagai objek Transaksi dan disimpan di dalam objek Rekening.

Penerapan relasi tersebut menggunakan struktur data dinamis ArrayList untuk mengelola kumpulan objek transaksi yang dimiliki oleh sebuah rekening. Dengan demikian, sistem tidak hanya dapat melakukan transaksi, tetapi juga menyimpan dan menampilkan riwayat transaksi yang telah dilakukan.

2. Tujuan

- Memahami dan menerjemahkan Class Diagram ke dalam implementasi program menggunakan bahasa Java.
- Memahami serta menerapkan konsep relasi Asosiasi one-to-many antarobjek.
- Mengimplementasikan pengelolaan kumpulan objek menggunakan ArrayList serta menerapkan pencatatan dan penampilan riwayat transaksi.

3. Manfaat

- Meningkatkan pemahaman mengenai hubungan antarclass dan objek dalam Pemrograman Berorientasi Objek.
- Melatih penggunaan ArrayList untuk menyimpan dan mengelola kumpulan objek secara dinamis.
- Memahami penerapan pencatatan riwayat transaksi sehingga aktivitas setor dan tarik tunai dapat direkam serta ditampilkan kembali kepada pengguna.

BAB II

Isi

1. Memastikan Akses Modifier di Rekening.java

```
private String nomorRekening;  
private String namaPemilik;  
private String pin;  
  
protected double saldo;  
protected ArrayList<Transaksi> riwayatTransaksi;
```

Bagian ini menentukan tingkat akses atribut di class Rekening. Tiga atribut pertama, yaitu nomorRekening, namaPemilik, dan pin, bersifat private, sehingga hanya bisa diakses dari dalam class Rekening itu sendiri. Ini sesuai prinsip information hiding: data sensitif (terutama PIN) tidak boleh dibaca atau diubah langsung dari luar, melainkan lewat method yang sudah disediakan seperti getter atau otentikasi(). Sementara itu, saldo dan riwayatTransaksi diberi protected, artinya masih terlindungi dari akses sembarangan di luar package, tetapi tetap bisa diakses oleh class dalam package yang sama dan oleh subclass (misalnya jenis rekening turunan dalam hierarki inheritance). Perlu dicatat, di modul bagian ini sebenarnya ditulis private untuk saldo dan riwayatTransaksi, jadi versi protected ini adalah variasi yang berguna bila nanti ada subclass yang perlu mengelola saldo dan riwayat secara langsung, dengan konsekuensi perlindungannya sedikit lebih longgar.

2. Membuat Subclass RekeningTabungan

```
package PraktikumMinggu4;  
  
public class RekeningTabungan extends Rekening{  
    private double sukuBunga;  
  
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga) {  
        super(nomor, nama, saldoAwal, pinAwal);  
        this.sukuBunga = sukuBunga;  
    }  
  
    public void tambahBungaAkhirBulan() {  
        double nominalBunga = saldo * (sukuBunga / 100);  
        saldo += nominalBunga;  
  
        String idTrx = "TRX-B-" + System.currentTimeMillis();  
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));  
    }  
}
```

```

        System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan :
Rp" + nominalBunga);
    }
}

```

Kode ini adalah contoh nyata kenapa saldo dan riwayatTransaksi di Rekening diberi protected. RekeningTabungan adalah subclass dari Rekening (lewat extends), jadi di method tambahBungaAkhirBulan() ia bisa langsung membaca dan mengubah saldo serta menambahkan data ke riwayatTransaksi tanpa getter atau setter. Seandainya keduanya private, baris saldo += nominalBunga dan riwayatTransaksi.add(...) akan error karena subclass tidak punya akses ke anggota private milik parent. Sebaliknya, sukuBunga dibuat private karena hanya relevan untuk RekeningTabungan sendiri. Constructor-nya memakai super(...) untuk meneruskan nomor, nama, saldo awal, dan PIN ke constructor Rekening, sehingga validasi PIN 6 digit tetap berjalan. Satu hal yang patut dicatat: karena protected juga bisa diakses class lain dalam package yang sama, saldo tetap bisa dimanipulasi dari luar subclass asalkan masih satu package (PraktikumMinggu4), jadi perlindungannya tidak seketat private.

3. Membuat Subclass RekeningGiro

```

package PraktikumMinggu4;

public class RekeningGiro extends Rekening{
    private double batasOverdraft;
    public RekeningGiro(String nomor, String nama, double saldoAwal, String
pinAwal, double batasOverdraft) {
        super(nomor,nama,saldoAwal,pinAwal);
        this.batasOverdraft = batasOverdraft;
    }
}

```

Kode ini adalah subclass lain dari Rekening, yaitu RekeningGiro, yang menambahkan satu atribut khusus bernama batasOverdraft dengan modifier private, sehingga hanya bisa diakses dari dalam class RekeningGiro sendiri (sama seperti sukuBunga di RekeningTabungan). Constructor-nya memakai super(nomor, nama, saldoAwal, pinAwal) untuk menyerahkan data umum ke constructor Rekening, jadi penyimpanan nomor, nama, saldo, dan validasi PIN 6 digit tetap

ditangani oleh parent, lalu batasOverdraft diisi setelahnya. Saat ini class ini belum punya method apa pun, sehingga batasOverdraft baru tersimpan dan belum dipakai. Karena saldo di parent berstatus protected, nantinya class ini bisa langsung mengakses saldo untuk logika overdraft, tetapi atribut batasOverdraft sendiri tetap tertutup dari luar kecuali ditambahkan getter atau setter.

BAB III

Latihan

1. Pembuatan Multi-Produk (40%)

```
case 1:
    System.out.println("Masukkan PIN awal:");
    String pin = input.nextLine();
    System.out.println("Masukkan saldo awal: ");
    double saldo = input.nextDouble();

    input.nextLine();
    System.out.println("Pilih opsi produk: 1. Tabungan umum
| 2. Giro bisnis");

    int pil = input.nextInt();
    input.nextLine();
    System.out.println("Masukkan no rekening: ");
    String no = input.nextLine();
    System.out.println("Masukkan nama pemilik: ");
    String nama = input.nextLine();

    if (saldo >= 50000 && pin.length() == 6) {
        if (pil == 1) {
            System.out.println("Input suku bunga");
            double sukuBunga = input.nextDouble();
            RekeningTabungan newRT = new
RekeningTabungan(no,nama, saldo, pin, sukuBunga);
            rekAr.add(newRT);
        }
        else if(pil == 2) {
            System.out.println("Input batas OverDraft
");
            double batas0 = input.nextDouble();
            RekeningGiro newGR = new
RekeningGiro(no,nama, saldo, pin, batas0);
            rekAr.add(newGR);
        }
    }
    else if(pin.length() != 6) {
        System.out.println("Pin harus sepanjang 6
angka");
    }
    else {
        System.out.println("Minimal saldo awal 50000");
    }
    break;
```

Kode ini adalah implementasi Menu 1 (Buka Rekening) yang meminta data dari user lalu membuat objek rekening sesuai jenis produk yang dipilih. Alurnya: program meminta PIN, saldo awal, pilihan produk (1 = Tabungan, 2 = Giro), nomor rekening, dan nama pemilik.

Pemanggilan `input.nextLine()` setelah `nextDouble()` dan `nextInt()` berfungsi membuang sisa karakter enter di buffer, supaya input String berikutnya tidak terlewat. Setelah itu ada validasi: jika saldo minimal 50000 dan panjang PIN tepat 6, program masuk ke percabangan produk. Pilihan 1 meminta suku bunga lalu membuat `RekeningTabungan`, sedangkan pilihan 2 meminta batas overdraft lalu membuat `RekeningGiro`, dan keduanya dimasukkan ke `rekAr`. Jika validasi gagal, else if mencetak pesan bahwa PIN harus 6 karakter, dan else mencetak pesan minimal saldo awal 50000. Sesuai tugas, PIN divalidasi sebelum objek dibuat, jadi constructor tidak perlu jatuh ke PIN default 123456.

Ada beberapa hal yang perlu dicatat. Pertama, `pin.length() == 6` hanya mengecek panjang, padahal tugas menyebut "String angka 6 digit", sehingga input seperti abcdef masih lolos. Kedua, jika pil bukan 1 atau 2 (misalnya 3), tidak ada pesan apa pun dan rekening tidak dibuat tanpa pemberitahuan. Ketiga, jika PIN salah dan saldo juga kurang dari 50000, hanya pesan PIN yang muncul karena else if dicek lebih dulu. Terakhir, tidak ada pesan konfirmasi saat rekening berhasil dibuat di bagian ini (kecuali yang dicetak constructor `Rekening`).

2. Simulasi Waktu Akhir Bulan : (60%)

```
case 6:
    if (akunAktif instanceof RekeningTabungan) {
        RekeningTabungan cAK = (RekeningTabungan)
        cAK.tambahBungaAkhirBulan();
    }
    else {
        System.out.println("Fitur Khusus Akun
        Tabungan");
    }
    break;
```

Kode ini adalah Menu 6 yang khusus memicu perhitungan bunga akhir bulan. `instanceof` mengecek apakah objek `akunAktif` (bertipe `Rekening`) sebenarnya adalah `RekeningTabungan`. Jika ya, objeknya di-cast ke `RekeningTabungan` supaya method `tambahBungaAkhirBulan()` bisa dipanggil, karena method itu hanya ada di subclass dan tidak dikenali oleh tipe `Rekening`. Jika bukan (misalnya `RekeningGiro`), program mencetak "Fitur Khusus Akun Tabungan". Pengecekan sebelum cast ini penting untuk mencegah `ClassCastException`. Perlu diperhatikan, pada modul tugas Menu 6 awalnya adalah Cetak Mutasi yang wajib meminta PIN lewat `otentikasi()`, sedangkan di kode ini menu 6 dipakai untuk fitur bunga dan belum ada pengecekan PIN sama sekali.

BAB IV

Kesimpulan

1. Kesimpulan

Praktikum ini menunjukkan bahwa enkapsulasi adalah fondasi untuk menjaga keamanan dan integritas data dalam program berorientasi objek. Dengan mengubah atribut sensitif seperti saldo, pin, nomorRekening, dan namaPemilik menjadi private, data tidak lagi bisa dimanipulasi langsung dari luar class (misalnya `akunAktif.saldo = 9999999`), melainkan hanya lewat method yang sudah dikontrol. Class Transaksi dibuat immutable dengan hanya menyediakan getter tanpa setter, sedangkan Rekening tidak menyediakan `setSaldo()` maupun `getPin()` agar saldo hanya berubah lewat transaksi yang sah dan PIN tidak pernah terbaca dari luar. Validasi PIN 6 digit di constructor serta method `otentikasi()` membuktikan bahwa enkapsulasi memungkinkan business logic ditanam di dalam class itu sendiri, bukan diserahkan ke kode pemanggil. Praktikum ini juga memperkenalkan peran `protected` pada hierarki inheritance: dengan saldo dan riwayatTransaksi berstatus `protected`, subclass seperti `RekeningTabungan` dan `RekeningGiro` dapat mengelolanya langsung (misalnya untuk menambah bunga) tanpa membuka aksesnya ke seluruh program, meskipun perlingkungannya sedikit lebih longgar daripada `private` karena class lain dalam package yang sama tetap bisa mengaksesnya. Pada sisi aplikasi, `Main.java` kemudian diperbarui agar memvalidasi PIN saat pembuatan rekening, memakai `instanceof` dan casting untuk fitur khusus tiap jenis rekening, serta (sesuai tugas) mewajibkan otentikasi sebelum transaksi sensitif. Secara keseluruhan, praktikum ini mengajarkan bahwa class yang baik harus menyembunyikan detail internalnya, mengekspos hanya behavior yang diperlukan, dan memastikan setiap perubahan state melewati aturan validasi yang konsisten.